STEC Immobilizer Gen. A

Diagnostic Specification

Confidential

Version: 1.0

Status:

Date: Sep-19,2007

Author:

Version	Date	Author	Description
1.0	Sep_19_07		

1 Introduction

This diagnostic specification is part of the functional description.

Additional details to the sections of the german implementation standard ISO 14230-3 (1994) are specified here. All project specific implemented services and parameter of diagnostic communication are described below.

Notice the duty of the supplier to document the development samples, see chapter "Development sample documentation".

Confidential

1.1 List of tables

Table 3.1: Service identifier value summary table 4
Table 8.1: Start communication request service 6
Table 8.2: Start communication positive response 6
Table 8.3: Stop communication request service 6
Table 8.4: Stop communication positive response 6
Table 8.5: Stop communication negative response 6
Table 9.1: Tester present request service Id 7
Table 9.2: Tester present positive response service Id 7
Table 9.3: Tester present negative response service Id 7
Table 9.4: Read ECU identification request service Id 7
Table 9.5: Definition of identification option values 7
Table 9.6: Read ECU identification positive response 8
Table 9.7: Read ECU identification negative response 8
Table 10.1: Read data by local identifier request service Id 9
Table 10.2: Definition of recordlocalIdentifier values 9
Table 10.3: Aftersales service record local identifier 9
Table 10.4: Statusbyte Input / Output signals , (Byte 10) 10
Table 10.5: Statusbyte Transponder , (Byte 11) 10
Table 10.6: Statusbyte position of last received transponder fixcode , (Byte 12) 10
Table 10.7: Statusbyte programmed positions of transponder fixcodes , (Byte 13) 11
Table 10.8: Statusbyte ECU challenge/response sequence (K-line disconnected) number 1 , (Byte 14) 11
Table 10.9: Statusbyte ECU challenge/response sequence (K-line disconnected) number 2 , (Byte 15) 11
Table 10.10: Statusbyte ECU challenge/response sequence (K-line disconnected) number 3 , (Byte 16) 12
Table 10.11: Statusbyte Programming, (Byte 17) 12
Table 10.12: Statusbyte Security code , (Byte 18) 13
Table 10.13: Statusbyte Negative Responsecodes received from ECM - Number 1 , (Byte 19) 13
Table 10.14: Statusbyte Negative Responsecodes received from ECM - Number 2 , (Byte 20) 13
Table 10.15: Reading the Mechanical Keynumber 14
Table 10.16: Read data by local identifier negativ response 14
Table 11.1: Read diagnostic trouble codes by status request Sid 15
Table 11.2: Read diagnostic trouble code by status positive response 15
Table 11.3: Read diagnostic trouble code by status negative response 15
Table 11.4: Supported DTC's The troublecode handling has to be equivalent to GME 1255 .....
2. Nevertheless this has to be discussed with the responsible system engineer. 16
Table 11.5: Clear diagnostic information request service Id 17
Table 11.6: Clear diagnostic information positive response Service Id 17
Table 11.7: Clear diagnostic information negative response 17
Table 12.1: Input output control by local identifier request service Id 18
Table 12.2: Input output control by local identifier positive response 18

Table 12.3: Input output control by local identifier negative response 18
Table 12.4: Definition of input output parameter values 19
Table 12.5: System programming 19
Table 12.6: Actuator tests 20
Table 12.7 Program Security Code 20
Table 12.8 Enter Security Code 20
Table 12.9 Program Transponder (Secret Key and Fixcode) 21
Table 12.10 Program Transponder Fixcode 21
Table 12.11 Erase Transponder Fixcode 22
Table 12.12 Transponder Fixcode Position 22
Table 12.13 Program Mechanical Keynumber 23
Table 12.14 Program Vehicle Identification Number 24
Table 12.15 Program repair shop code 25
Table 12.16 Mask /Demask Troublecodes 26
Table 12.17 Byte LED-Output 26
Table 12.18 Reset Immobiliser 27
Table 12.19 Reset ECM 27
Table 12.20 Program Secret Key and Security Code into ECM 28
Table 12.21 Program Secret Key from ECM 28
Table 12.22 Actuator test - LED-Output (Activate Actuatortest) 29
Table 12.23 Byte LED-Output 29
Table 12.24 Actuator test - LED-Output (Return_Control_to_ECU) 29
Table 12.28 Actuator test - Read Transponder (Activate Actuatortest) 30
Table 12.29 Byte Read Transponder 30
Table 12.30 Actuator test - Read Transponder (Return_Control_to_ECU) 30

Confidential

2 Service identifier value summary table

The left column of the following table lists all implemented services of the electronic control unit, the 2nd column specifies the mnemonic of the service, the 3rd column assigns the keyword protocol 2000 implementation „request hex value“, the 4th column assigns the keyword protocol 2000 implementation „positive response hex value“, the 5th column indicates if a negative response message (serviceId = $7F) shall be supported by the electronic control unit.

Table 2.1: Service identifier value summary table

diagnostic service name	mnemonic	request hex value	response hex value	negative response
ClearDiagnosticInformation	CDI	14	54	7F
InputOutputControlByLocalIdentifier	IOCBLI	30	70	7F
ReadDataByLocalIdentifier	RDBLI	21	61	7F
ReadDiagnosticTroubleCodesByStatus	RDTCBS	18	58	7F
ReadEcuIdentification	REI	1A	5A	7F
StartCommunication	STC	81	C1	no
StopCommunication	SPC	82	C2	7F
TesterPresent	TP	3E	7E	7F

Confidential

3 Definitions

3.1 Target and source address

Target address for immobilizer ECU to tester:	C0 hex
Target address for immobilizer ECU to EMS:	C1 hex
Target address for EMS to tester and immobilizer:	11 hex
Source address for client (tester):	F0 hex ..... FD hex

4 Negative Responsecodes

All negative responsecodes used by the immobiliser ECU are listed at the end of this document (see chapter 16). The responsecode 21 h (Busy repeat request) shall not be used by the immobiliser ECU.

5 Error-Handling

If an invalid message is detected caused by a checksum error, byte error, overrun condition or any other interference on the physical or data link layer, the system shall not respond to the request. Thus an intertime out at P2 is generated and the tester will repeat the request.

For invalid messages on the implementation layer the system shall respond with a negative response in order to keep the communication running and to inform the client (with the negative responsecode) about the current error situation.

Confidential

6 Communication services

6.1 Start communication

Table 6.1: Start communication request service

data byte	parameter name	Cvt	hex value	mnemonic
#1	**startCommunication Request Service Id**	**M**	**81**	**STC**

Table 6.2: Start communication positive response

data byte	parameter name	Cvt	hex value	mnemonic
#1	**startCommunication Positive Response Service Id**	**S**	**C1**	**STCPR**
#2 #3	Keybyte 1 Keybyte 2	M	E5 8F	KB1 KB2

Note: The „Start_Communication Service“ will be handled with the 3-Byte Format (Source and Target Information). The following communication between Tester and Immobiliser ECU will be handled with 1- Byte Format. The Standard Timing parameters will be used.

After the „Start_Communication_Service“ the immobiliser will check the formatbyte of the tester request. In case the immobiliser detects that the used format is incorrect (either bit A0 or bit A1 of the format byte have a value of „Log 1“) it doesn´t respond to this request and waits for another request.

6.2 Stop communication

Table 6.3: Stop communication request service

data byte	parameter name	Cvt	hex value	mnemonic
#1	**stopCommunication Request Service Id**	**M**	**82**	**SPC**

Table 6.4: Stop communication positive response

data byte	parameter name	Cvt	hex value	mnemonic
#1	**stopCommunication Positive Response Service Id**	**S**	**C2**	**SPCPR**

Table 6.5: Stop communication negative response

data byte	parameter name	Cvt	hex value	mnemonic
#1	**negative Response Service Id**	**S**	**7F**	**NR**
#2	**stopCommunication Request Service Identification**	**S**	**82**	**SCR**
#3	**ResponseCode**	**M**	**xx**	**RC**

Confidential

7 Diagnostic management functional unit

7.1 Tester present

Table 7.1: Tester present request service Id

data byte	parameter name	Cvt	hex value	mnemonic
#1	**testerPresent Request Service Id**	**M**	**3E**	**TP**

Table 7.2: Tester present positive response service Id

data byte	parameter name	Cvt	hex value	mnemonic
#1	**testerPresent Positive Response Service Id**	**S**	**7E**	**TPPR**

Table 7.3: Tester present negative response service Id

data byte	parameter name	Cvt	hex value	mnemonic
#1	**negativeResponse Service Id**	**S**	**7F**	**NR**
#2	**testerPresent Request Service Id**	**M**	**3E**	**TP**
#3	responseCode	M	xx	**RC_...**

7.2 Read ECU identification

Table 7.4: Read ECU identification request service Id

Data byte	parameter name	Cvt	hex value	mnemonic
#1	**readEcuIdentification Request Service Id**	**M**	**1A**	**REI**
#2	identificationOption	M	81-9B	**IO_...**

Table 7.5: Definition of identification option values

Hex value	ECU identification option description (IO_...)	ECU identification data example	Scaling data	data length (Byte)	scaling type
81	ECU identification scaling table (use of it depends on the tester concept)	04 90 6F ...	04 90 6F ...	26	
90	Vehicle identification number (VIN)	t.b.d	04 90 6F 62	17	ASCII
91	Part number	e.g. 13 144 390	03 91 6B 03	11	ASCII
92	ECU hardware number (KW/Year)	e.g. 51 02	03 92 02	2	HEX
94	ECU software number (KW/Year)	e.g. 04 03	03 94 02	2	HEX
96	Type approval number (use of it depends on the OEM concept)	t.b.d	03 96 6A	10	ASCII
98	Repair shop code or tester serial number (10 Byte , ASCII)	e.g. 6359984	03 98 6A	10	ASCII
9A	ECU diagnostic data identifier	01;01	03 9A 02	2	HEX
9B	Date of production (Year/KW)	e.g. 2003 25	03 9B 66	6	ASCII

Confidential

Table 7.6: Read ECU identification positive response

data byte	parameter name	Cvt	hex value	mnemonic
#1	**readEcuIdentification Positive Response Service Id**	**S**	**5A**	**REIPR**
	identificationRecordValue=[		xx=[	**IRV_...**
#2	identificationOption	M	81-9B,	**IO_...**
#3	ECUIdentificationParameter#1	C	xx,	**ECUIP_...**
:	**:**	**:**	**:**	**:**
#n	ECUIdentificationParameter#m	C	xx	**ECUIP_...**
	]		]	

Table 7.7: Read ECU identification negative response

data byte	parameter name	Cvt	hex value	mnemonic
#1	**negativeResponse Service Id**	**S**	**7F**	**NR**
#2	**readEcuIdentification Request Service Id**	**M**	**1A**	**REI**
#3	responseCode	M	xx	**RC_...**

Confidential

8 Data transmission functional unit

8.1 Read data by local identifier

Table 8.1: Read data by local identifier request service Id

data byte	parameter name	Cvt	hex value	mnemonic
#1	**readDataByLocalIdentifier Request Service Id**	**M**	**21**	**RDBLI**
#2	recordlocalIdentifier (See Table 8.2)	M	xx	**RLI_...**

The parameter readDataByLocalIdentifierID identifies the parameters that shall be read:

Table 8.2: Definition of recordlocalIdentifier values

hex value	Description
01	**Aftersales Service record local identifier** This parameter identifies the Aftersales service record local identifier (Snapshot) to be read out of the immobiliser ECU
03	**Mechanical Keynumber** This parameter identifies the Mechanical Keynumber to be read out of the immobiliser ECU.

8.1.1 Aftersales service record local identifier

Table 8.3: Aftersales service record local identifier

data byte	parameter name	format and remark	#pin ECU	Cvt	hex value	mnemonic
#1	**ReadDataByLocalIdentifier Positive Response Service Id**	**-**	**-**	**M**	**61**	**RDBLIPR**
#2	Record local identifier	-	-	M	01	**RLI_...**
#3	Number of DTC	-	-	M	xx	**RV_...**
#4	DTC#1	-	-	M	xx	
#5	DTC#1	-	-	M	xx	
#6	Status of DTC#1	-	-	M	xx	
#7	DTC#2	-	-	M	xx	
#8	DTC#2	-	-	M	xx	
#9	Status of DTC#2	-	-	M	xx	
#10	Statusbyte input / output signals	see Table 8.4	-	M	xx	
#11	Statusbyte Transponder	see Table 8.5	-	M	xx	
#12	Statusbyte position of last received transponder fixcode	see Table 8.6	-	M	xx	
#13	Statusbyte programmed positions of transponder fixcode	see Table 8.7	-	M	xx	
#14	Statusbyte ECU challenge/response sequence (K-line disconnected) number 1	see Table 8.8	-	M	xx	
#15	Statusbyte ECU challenge/response sequence (K-line disconnected) number 2	see Table 8.9	-	M	xx	
#16	Statusbyte ECU challenge/response sequence (K-line disconnected) number 3	see Table 8.10	-	M	xx	
#17	Statusbyte Programming	see Table 8.11	-	M	xx	
#18	Statusbyte Security Code	see Table 8.12	-	M	xx	

Confidential

# 19	Statusbyte NR-Codes received from ECM / number 1	see Table 8.13		M	xx	
#20	Statusbyte Statusbyte NR-Codes received from ECM / number 2	see Table 8.14	-	M	xx	

Table 8.4: Statusbyte Input / Output signals , (Byte 10)

Bit	log	description text	remark	#pin ECU
0	0 1	Ignition off Ignition on	ground battery	Pin 5
1	0 1	R-Line off R-Line on	battery ground	Pin 2
2	0 1	LED physically low LED physically high	This bit indicates the status of the feedback physical output value	Pin 1
3	0	Output reserve	TBD	TBD
4	0 1	LED output inactive LED output active	active - low	Pin1
5…7	0	Output reserve	TBD	TBD

Table 8.5: Statusbyte Transponder , (Byte 11)

bit	log	description text	remark
0	0 1	No Transponder detected Transponder detected	
1	0 1	No STEC Transponder detected STEC Transponder detected	
2	0	Reserve	
3	0 1	No Krypto-Transponder detected Krypto-Transponder detected	
4	0 1	No Krypto-Transponder in delivery condition detected Krypto-Transponder in delivery condition detected	
5	0 1	No Programmed Krypto-Transponder detected Programmed Krypto-Transponder detected	Transponder was programmed by an ECU before
6…7	0	Reserve	TBD

Table 8.6: Statusbyte position of last received transponder fixcode , (Byte 12)

bit	log	Description text	remark
0	0 1	Last received transponder fixcode is not programmed at a transponder position Last received transponder fixcode is programmed at a transponder position	-
1	0 1	Last received transponder fixcode is not programmed at transponder position 1 Last received transponder fixcode is programmed at transponder position 1	-
2	0 1	Last received transponder fixcode is not programmed at transponder position 2 Last received transponder fixcode is programmed at transponder position 2	-
3	0 1	Last received transponder fixcode is not programmed at transponder position 3 Last received transponder fixcode is programmed at transponder position 3	-
4	0 1	Last received transponder fixcode is not programmed at transponder position 4 Last received transponder fixcode is programmed at transponder position 4	-
5	0 1	Last received transponder fixcode is not programmed at transponder position 5 Last received transponder fixcode is programmed at transponder position 5	-

Confidential

6…7	0	Reserve	TBD

Table 8.7: Statusbyte programmed positions of transponder fixcodes , (Byte 13)

bit	log	Description text	remark
0	0 1	No transponder fixcode is programmed At least one transponder fixcode is programmed	-
1	0 1	Transponder fixcode at transponder position 1 is not programmed Transponder fixcode at transponder position 1 is programmed	-
2	0 1	Transponder fixcode at transponder position 2 is not programmed Transponder fixcode at transponder position 2 is programmed	-
3	0 1	Transponder fixcode at transponder position 3 is not programmed Transponder fixcode at transponder position 3 is programmed	-
4	0 1	Transponder fixcode at transponder position 4 is not programmed Transponder fixcode at transponder position 4 is programmed	-
5	0 1	Transponder fixcode at transponder position 5 is not programmed Transponder fixcode at transponder position 5 is programmed	-
6…7	0	Reserve	TBD

Table 8.8: Statusbyte ECU challenge/response sequence (K-line disconnected) number 1 , (Byte 14)

bit	log	Description text	remark
0	0	Reserve	-
1	0	Reserve	
2	0 1	Correct ECM request signal not received Correct ECM request signal received	-
3	0 1	Errorhandling active during Challenge/Response sequence with ECM No Errorhandling active during Challenge/Response sequence with ECM	This Bit indicates as a general information whether or not an Errorhandling occured during the challenge/response sequence with the ECM
4…7	0	Reserve	

Table 8.9: Statusbyte ECU challenge/response sequence (K-line disconnected) number 2 , (Byte 15)

bit	log	Description text	remark
0	0 1	No Start_Communication_Request message transmitted Start_Communication_Request message transmitted	Start_Communication_Service (Request includes Wake_up Pattern)
1	0 1	Errorhandling active during Start_Communication_Service No Errorhandling active during Start_Communication_Service	Start_Communication_ Service
2	0 1	Start_Communication_Service not completed Start_Communication_Service completed	Start_Communication_ Service
3	0 1	No Immo_Challenge_Request message transmitted Immo_Challenge_Request message transmitted	Immo_Challenge_Service
4	0 1	Errorhandling active during Immo_Challenge_Service No Errorhandling active during Immo_Challenge_Service	Immo_Challenge_Service
5	0 1	Immo_Challenge_Service not completed Immo_Challenge_Service completed	Immo_Challenge_Service
6	0 1	Errorhandling active during Transponder_Communication No Errorhandling active during Transponder_Communication	„No Acknowledge“ received during challenge/response sequence with transponder
7	0 1	Transponder_Communication not correct Transponder_Communication correct	Challenge/Response sequence with Transponder

Note: For each service of the communication between Immobiliser ECU and engine control module three statusbits are used.

The first bit indicates whether or not the service request was transmitted to the engine control module.

The second bit indicates whether or not an Errorhandling occured within this service. The information „no

Confidential

Errorhandling active during Service“ is only displayed if a positive response has been received with the first request.

The third bit indicates whether or not the service could be completed with the Errorhandling. The information “Service Completed“ is only displayed if a positive response has been received either with the first request or within the errorhandling.
In case the immobiliser receives a specific negative responsecode (e.g. immo_function_not_programmed) the status will indicate an active errorhandling and an incompleted service. In this case table 10.13 has to be used for further information.

Table 8.10: Statusbyte ECU challenge/response sequence (K-line disconnected) number 3 , (Byte 16)

bit	log	Description text	remark
0	0 1	No Immo_Code_Request message transmitted Immo_Code_Request message transmitted	Immo_Code_Service
1	0 1	Errorhandling active during Immo_Code_Service No Errorhandling active during Immo_Code_Service	Immo_Code_Service
2	0 1	Immo_Code_Service not completed Immo_Code_Service completed	Immo_Code_Service
3	0 1	No Stop_Communication_Request message transmitted Stop_Communication_Request message transmitted	Stop_Communication_Service
4	0 1	Errorhandling active during Stop_Communication_Service No Errorhandling active during Stop_Communication_Service	Stop_Communication_Service
5	0 1	Stop_Communication_Service not completed Stop_Communication_Service completed	Stop_Communication_ Service
6	0 1	Within Challenge/Response sequence a connected Tester was not recognised on K-Line Within Challenge/Response sequence a connected Tester was recognised on K-Line	Tester recognised on K-Line during Challenge/Response Sequence
7	0	Reserve	

Table 8.11: Statusbyte Programming, (Byte 17)

bit	log	Description text	remark
0	0 1	ECU is in delivery condition ECU is not in delivery condition	This bit will be reset, if one of the following programming actions (see Table 10.5, programming steps number 1, 3, 4, 6, 7, 8, 9, 12, 13) was performed.
1	0 1	Vehicle Identification number is not programmed Vehicle Identification number is programmed	-
2	0 1	Mechanical Keynumber is not programmed Mechanical Keynumber is programmed	-
3	0 1	Repair shop code or tester serial number is not programmed Repair shop code or tester srial number is programmed	-
4	0 1	Trouble codes for LED-Output are masked Trouble codes for LED-Output are not masked	-
5…7	0	Reserve	-

Confidential

Table 8.12: Statusbyte Security code , (Byte 18)

bit	log	Description text	remark
0	0 1	Security code is not programmed Security code is programmed	-
1	0 1	Wait-loop is inactive, the Security code can be entered Wait-loop is active, the Security code can´t be entered	-
2	0 1	The Security code isn´t entered The correct Security code is entered	-
3	0	Reserve	TBD
  4 5 6 7	 - - - -	Number of posibilities to enter the correct security code (example 1) Position 1 (example 1) Position 2 (example 0) Position 3 (example 0) Position 4 (example 0)	binary, range 10 to 0

Table 8.13: Statusbyte Negative Responsecodes received from ECM - Number 1 , (Byte 19)

bit	log	Description text	remark
0	0 1	KW 2000 NR-Code not received KW 2000 NR-Code received (one of the below listed or general KW 2000 NR-Code)	
1	0 1	NR-Code „pre-release_time_expired“ not received NR-Code „pre-release_time_Expired“ received	
2	0 1	NR-Code „ subfunction_not_supported“ not received NR-Code „ subfunction_not_supported“ received	
3	0 1	NR-Code „system_not_in_delivery_condition“ not received NR-Code „system_not_in_delivery_condition“ received	
4	0 1	NR-Code „incorrect_security_code“ not received NR-Code „incorrect_security_code“ received	
5	0 1	NR-Code „required_time_delay_not_expired“ not received NR-Code „required_time_delay_not_expired“ received	
6	0 1	NR-Code „download_not_accepted“ not received NR-Code „download_not_accepted“ received	
7	0 1	NR-Code „immo_function_not_programmed“ not received NR-Code „immo_function_not_programmed“ received	

Table 8.14: Statusbyte Negative Responsecodes received from ECM - Number 2 , (Byte 20)

bit	log	Description text	remark
0	0 1	NR-Code „response_not_yet_generated“ not received NR-Code „response_not_yet_generated“ received	
1	0 1	NR-Code „transponder_response_incorrect“ not received NR-Code „transponder_response_incorrect“ received	
2…7	0	Reserve	TBD

Confidential

8.1.2 Read Mechanical Keynumber

Reading the Mechanical Keynumber is realised as shown below in Table 8.15

Table 8.15: Reading the Mechanical Keynumber

data byte	parameter name	format and remark	#pin ECU	Cvt	hex value	mnemonic
#1	**ReadDataByLocalIdentifier Positive Response Service Id**	**-**		**M**	**61**	**RDBLIPR**
#2	Mechanical Keynumber		-	M	03	**.**
#3	Mechanical Keynumber Byte 1	ASCII	-	M	xx	
#3	Mechanical Keynumber Byte 2	ASCII	-	M	xx	
#3	Mechanical Keynumber Byte 3	ASCII	-	M	xx	
#3	Mechanical Keynumber Byte 4	ASCII	-	M	xx	
#3	Mechanical Keynumber Byte 5	ASCII	-	M	xx	

Table 8.16: Read data by local identifier negativ response

data byte	parameter name	Cvt	hex value	mnemonic
#1	**NegativeResponse Service Id**	**S**	**7F**	**NR**
#2	**ReadDataByLocalIdentifier Request Service Id**	**M**	**21**	**RDBLI**
#3	ResponseCode	M	xx	**RC_...**

Confidential

9 Stored data transmission function unit

9.1 Read diagnostic trouble code by status

Table 9.1: Read diagnostic trouble codes by status request Sid

data byte	parameter name	Cvt	hex value	mnemonic
#1	**ReadDiagnosticTroubleCodesByStatus Request SId**	**M**	**18**	**RDTCBS**
#2	StatusOfDTC	M	00	**SODTC_...**
#3 #4	GroupOfDTC = requestByFunction (High Byte) GroupOfDTC = requestByFunction (Low Byte)	M M	80 or FF 00	**GODTC_...**

Note:The High Byte of groupOfDTC can either have the value 80 h (Body group) or FF h (All groups).

Table 9.2: Read diagnostic trouble code by status positive response

data byte	parameter name	Cvt	hex value	mnemonic
#1	**ReadDiagnosticTroubleCodesByStatus Pos. Response SId**	**M**	**58**	**RDTCBSPR**
#2	NumberOfDTC	M	xx	**NRODTC_...**
#3 : : : : : #n	ListOfDTCAndStatus=[DTC#1 (High Byte) DTC#1 (Low Byte) statusOfDTC#1 : DTC#m (High Byte) DTC#m (Low Byte) statusOfDTC#m]	C	xx xx xx : xx xx xx	**LODTCAS_...** **DTC_** **SODTC_** : **DTC_** **SODTC_**

Table 9.3: Read diagnostic trouble code by status negative response

data byte	parameter name	Cvt	hex value	mnemonic
#1	**NegativeResponse Service Id**	**S**	**7F**	**NR**
#2	**ReadDiagnosticTroubleCodesByStatus Request SId**	**M**	**18**	**RDTCBS**
#3	ResponseCode	M	xx	**RC_...**

Confidential

Table 9.4: Supported DTC's

DTC hex values	description text	History YES	History NO	output order
9000 hex	ECU defect, Internal errors		X	**1**
B040 hex	Communication error on W-Line, ECM doesn´t answer on Challenge/Response requests		X	2
B042 hex	W-Line short circuit to ground	X		6
B043 hex	W-line short circuit to battery	X		7
B045 hex	LED line short circuit to ground or open circuit, LED line malfunction	X		8
B048 hex	LED line short circuit to battery	X		9
B055 hex	No transponder modulation or no transponder	X		11
B056 hex	No transponder fixcode programmed		X	3
B057 hex	No security code programmed		X	4
B059 hex	No request from electronic engine control received		X	5
B060 hex	Unprogrammed transponder fixcode received	X		12
B061 hex	Disturbed or no challenge / response transponder communication	X		10

NOTE :

The DTC´s are output in the above mentioned ´output order´. But only a maximum of 7 DTC´s are supported at the same time.

The DTC ´9000hex - ECU defect, internal errors´ is always output first.

Confidential

9.2 Clear diagnostic information

Table 9.5: Clear diagnostic information request service Id

data byte	parameter name	Cvt	hex value	mnemonic
#1	**ClearDiagnosticInformation Request Service Id**	**M**	**14**	**CDI**
#2 #3	GroupOfDiagnosticInformation { High Byte } GroupOfDiagnosticInformation { Low Byte }	M M	80 or FF 00	**GODIN**

Note:The High Byte of groupOfDTC can either have the value 80 h (Body group) or FF h (All groups).

Table 9.6: Clear diagnostic information positive response Service Id

data byte	parameter name	Cvt	hex value	mnemonic
#1	**ClearDiagnosticInformation Positive Response SId**	**S**	**54**	**CDIPR**
#2 #3	GroupOfDiagnosticInformation { High Byte } GroupOfDiagnosticInformation { Low Byte }	M M	80 or FF 00	**GODIN**

Note:The High Byte of groupOfDTC can either have the value 80 h (Body group) or FF h (All groups).

Table 9.7: Clear diagnostic information negative response

data byte	parameter name	Cvt	hex value	mnemonic
#1	**NegativeResponse Service Id**	**S**	**7F**	**NR**
#2	**ClearDiagnosticInformation Request Service Id**	**M**	**14**	**CDI**
#3	ResponseCode	M	xx	**RC**

Confidential

10 Input output control functional unit

10.1 Input output control by local identifier

Table 10.1: Input output control by local identifier request service Id

data byte	parameter name	Cvt	hex value	mnemonic
#1	**InputOutputControlByLocalIdentifier Request SId**	**M**	**30**	**IOCBLI**
#2	InputOutputLocalIdentifier	M	xx	**IOLI_...**
#3 #4 : #n	ControlOption=[inputOutputControlParameter controlState#1 : controlState#m]	M C : C	xx xx : xx	**CO_ IOCP_...** **CS_...** : **CS_...**

Table 10.2: Input output control by local identifier positive response

data byte	parameter name	Cvt	hex value	mnemonic
#1	**InputOutputControlByLocalIdentifier Positive Response SId**	**S**	**70**	**IOCBLIPR**
#2	InputOutputLocalIdentifier	M	xx	**IOLI_...**
#3 #4 : #n	ControlOption=[inputOutputControlParameter controlState#1 : controlState#m]	M C : C	xx xx : xx	**CO_ IOCP_...** **CS_...** : **CS_...**

Table 10.3: Input output control by local identifier negative response

data byte	parameter name	Cvt	hex value	mnemonic
#1	**NegativeResponse Service Id**	**S**	**7F**	**NR**
#2	**InputOutputControlByLocalIdentifier Request SId**	**M**	**30**	**IOCBLI**
#3	ResponseCode	M	xx	**RC_...**

Confidential

Table 10.4: Definition of input output parameter values

hex	description	mnemonic
00	ReturnControlToECU This value shall indicate to the server (ECU) that the client (tester) does no longer have control about the input signal, internal parameter or output signal referenced by the inputOutputLocalIdentifier.	RCTECU
07	ShortTermAdjustment This value shall indicate to the server (ECU) that it is requested to adjust the input signal,internal parameter or output signal referenced by the inputOutputLocalIdentifier in RAM to the value(s) included in the controlOption parameter(s). (e.g. set Idle Air Control Valve to a specific step number, set puls width of valve to a specific value/duty cycle.	STA
08	LongTermAdjustment This value shall indicate to the server (ECU) that it is requested to adjust the input signal,internal parameter or output signal referenced by the inputOutputLocalIdentifier in EEPROM/FLASH EEPROM to the value(s) included in the controlOption parameter(s). (e.g. set Engine Idle Speed, set CO Adjustement parameter to the specific value.	LTA
0A	ECULernSecretKeyAndSecurityCode	ELSKASC
0B	ImmoLernSecretKey	ILSK
0E	ResetECUToDeliveryCondition	RETDC
0F	ResetImmobiliserToDeliveryCondition	RITDC

The following tables define the required system programming functions and required actuator tests. These functions have to be implemented within the „Input output control by local identifier service“ (see also Immo II with Keyword Protocoll 2000).

Table 10.5: System programming

No.	Programming Function description	see
1	Program Security Code	Table 10.7
2	Enter Security Code	Table 10.8
3	Program Transponder (Fixcode and Secret Key)	Table 10.9
4	Program Transponder (Fixcode)	Table 10.10
5	Erase Transponder (Fixcode)	Table 10.11 and Table 10.12
6	Program Mechanical Keynumber	Table 10.13
7	Program Vehicle Identification number	Table 10.14
8	Program Repair shop /Tester serial number (10 Bytes ASCII)	Table 10.15
9	Mask / Demask Troublecodes	Table 10.16 and Table 10.17
10	Reset Immo	Table 10.18
11	Reset ECM	Table 10.19
12	Program Secret Key and Security Code into ECM	Table 10.20
13	Program Secret Key from ECM	Table 10.21

Confidential

Table 10.6: Actuator tests

Number	Actuator test description	see
1	LED - Output	Table 10.22 , Table 10.23 and Table 10.24
2	Reserve	
3	Read Transponder and actualize snapshot	Table 10.25 , Table 10.26 and Table 10.27

Note: The Security Code protection of each of the above listed functions has to be equivalent to the concept used within the Immo 1 - ECU. Nevertheless this has to be discussed with the responsible system engineer.

Note: The actuator tests will be ended by the „return of control to ECU“ request , end of diagnose , disconnection of tester etc. The system condition before and after the actuator test has to be the same.

Table 10.7 Program Security Code

time	Client Request Message	Hex
P3	Format Byte	xx
	inputOutputControlByLocalId.Request[inputOutputLocalId = "securityCode" inputOutputControlParameter = longTermadjust securityCode { Byte 1 } securityCode { Byte 2 }	**30** 62 08 xx xx
	Checksum	xx

time	Server Positive Response Message	Hex
P2	Format Byte	xx
	InputOutputControlByLocalId.PosRsponse[inputOutputLocalId = "securityCode" inputOutputControlParameter = longTermadjust]	**70** 62 08
	Checksum	xx

Server Negative Response Message	Hex
Format Byte	xx
negativeResponse Service Identifier **inputOutputControlByLocalId.ReqSId** [responseCode]	**7F** **30** xx
Checksum	xx

Table 10.8 Enter Security Code

time	Client Request Message	Hex
P3	Format Byte	xx
	inputOutputControlByLocalId.Request[inputOutputLocalId = "securityCode" inputOutputControlParameter = short Termadjust securityCode { Byte 1 } securityCode { Byte 2 }	**30** 62 07 xx xx
	Checksum	xx

time	Server Positive Response Message	Hex
P2	Format Byte	xx
	InputOutputControlByLocalId.PosRsponse[InputOutputLocalId = "securityCode" InputOutputControlParameter = shortTermadjust]	**70** 62 07
	Checksum	xx

Server Negative Response Message	Hex
Format Byte	xx
negativeResponse Service Identifier **inputOutputControlByLocalId.ReqSId** [responseCode]	**7F** **30** xx
Checksum	xx

Confidential

Table 10.9 Program Transponder (Secret Key and Fixcode)

time	Client Request Message	Hex
P3	Format Byte	xx
	inputOutputControlByLocalId.Request[inputOutputLocalId = "Transponder Secret Key and Fixcode" inputOutputControlParameter = long Termadjust	**30** 03 08
	Checksum	xx

time	Server Positive Response Message	Hex	Server Negative Response Message	Hex
P2	Format Byte	xx	Format Byte	xx
	InputOutputControlByLocalId.PosRsponse[inputOutputLocalId = "Transponder Fixcode and Secret Key" inputOutputControlParameter = longTermadjust]	**70** 03 08	**negativeResponse Service Identifier** **inputOutputControlByLocalId.ReqSId** [responseCode]	**7F** **30** xx
	Checksum	xx	Checksum	xx

Table 10.10 Program Transponder Fixcode

time	Client Request Message	Hex
P3	Format Byte	xx
	inputOutputControlByLocalId.Request[inputOutputLocalId = "ProgramTransponder Fixcode" inputOutputControlParameter = long Termadjust	**30** 04 08
	Checksum	xx

time	Server Positive Response Message	Hex	Server Negative Response Message	Hex
P2	Format Byte	xx	Format Byte	xx
	InputOutputControlByLocalId.PosRsponse[inputOutputLocalId = "ProgramTransponder Fixcode " inputOutputControlParameter = longTermadjust]	**70** 04 08	**negativeResponse Service Identifier** **inputOutputControlByLocalId.ReqSId** [responseCode]	**7F** **30** xx
	Checksum	xx	Checksum	xx

Note: Within the functions „Program Transponder (Secret Key and Fixcode)“ or „Program Transponder Fixcode“ the next available transponder position is programmed automatically.

In order to provide a transponder programming history function the immobiliser stores the last position of programmed transponder fixcodes through an internal pointer even if a transponder has been erased.

Therefore the five transponder positions are handled like a „loop“ or a „cycle“ memory. If the pointer is set to e.g. position five the next transponder will be stored to position one (if this position is not programmed).

Table 10.11 Erase Transponder Fixcode

time	Client Request Message	Hex
P3	Format Byte	xx
	inputOutputControlByLocalId.Request[	**30**
	inputOutputLocalId = "EraseTransponder Fixcode"	05
	inputOutputControlParameter = long Termadjust	08
	Transponder Fixcode Position	00-05
	Checksum	xx

time	Server Positive Response Message	Hex	Server Negative Response Message	Hex
P2	Format Byte	xx	Format Byte	xx
	InputOutputControlByLocalId.PosRsponse[	**70**	**negativeResponse Service Identifier**	**7F**
	InputOutputLocalId = "EraseTransponder Fixcode "	05	**inputOutputControlByLocalId.ReqSId** [	**30**
	InputOutputControlParameter = longTermadjust]	08	responseCode]	xx
	Checksum	xx	Checksum	xx

Table 10.12 Transponder Fixcode Position

Hex value	description
00	Delete all programmed transponder fixcodes
01	Delete transponder fixcode at position 1
02	Delete transponder fixcode at position 2
03	Delete transponder fixcode at position 3
04	Delete transponder fixcode at position 4
05	Delete transponder fixcode at position 5

Confidential

Table 10.13 Program Mechanical Keynumber

time	Client Request Message	Hex
P3	Format Byte	xx
	inputOutputControlByLocalId.Request[	**30**
	inputOutputLocalId = "Program mechanical keynumber"	06
	inputOutputControlParameter = long Termadjust	08
	Byte 1	ASCII
	Byte 2	ASCII
	Byte 3	ASCII
	Byte 4	ASCII
	Byte 5	ASCII
	Checksum	xx

time	Server Positive Response Message	Hex	Server Negative Response Message	Hex
P2	Format Byte	xx	Format Byte	xx
	inputOutputControlByLocalId.PosRsponse	**70**	**negativeResponse Service Identifier**	**7F**
	[inputOutputLocalId = "Program mechanical keynumber "	06	**inputOutputControlByLocalId.ReqSId** [	**30**
	inputOutputControlParameter = longTermadjust]	08	responseCode]	xx
	Checksum	xx	Checksum	xx

Confidential

Table 10.14 Program Vehicle Identification Number

time	Client Request Message	Hex
P3	Format Byte	xx
	inputOutputControlByLocalId.Request[	**30**
	inputOutputLocalId = "Program vehicle identification number"	90
	inputOutputControlParameter = long Termadjust	08
	Byte 1	ASCII
	Byte 2	ASCII
	Byte 3	ASCII
	Byte 4	ASCII
	Byte 5	ASCII
	Byte 6	ASCII
	Byte 7	ASCII
	Byte 8	ASCII
	Byte 9	ASCII
	Byte 10	ASCII
	Byte 11	ASCII
	Byte 12	ASCII
	Byte 13	ASCII
	Byte 14	ASCII
	Byte 15	ASCII
	Byte 16	ASCII
	Byte 17	ASCII
	Checksum	xx

time	Server Positive Response Message	Hex
P2	Format Byte	xx
	inputOutputControlByLocalId.PosRsponse	**70**
	[inputOutputLocalId = "Program vehicle Identification number“	90
	inputOutputControlParameter = longTermadjust]	08
	Checksum	xx

Server Negative Response Message	Hex
Format Byte	xx
negativeResponse Service Identifier	**7F**
inputOutputControlByLocalId.ReqSId	**30**
[responseCode]	xx
Checksum	xx

Confidential

Table 10.15 Program repair shop code

time	Client Request Message	Hex
P3	Format Byte	xx
	inputOutputControlByLocalId.Request[	**30**
	inputOutputLocalId = "Program repair shop code"	98
	inputOutputControlParameter = long Termadjust	08
	Byte 1	ASCII
	Byte 2	ASCII
	Byte 3	ASCII
	Byte 4	ASCII
	Byte 5	ASCII
	Byte 6	ASCII
	Byte 7	ASCII
	Byte 8	ASCII
	Byte 9	ASCII
	Byte 10	ASCII
	Checksum	xx

time	Server Positive Response Message	Hex	Server Negative Response Message	Hex
P2	Format Byte	xx	Format Byte	xx
	inputOutputControlByLocalId.PosRspons	**70**	**negativeResponse Service Identifier**	**7F**
	[inputOutputLocalId = "Program repair shop code“	98	**inputOutputControlByLocalId.ReqSId** [	**30**
	inputOutputControlParameter = longTermadjust]	08	responseCode]	xx
	Checksum	xx	Checksum	xx

Confidential

Table 10.16 Mask /Demask Troublecodes

time	Client Request Message	Hex
P3	Format Byte	xx
	inputOutputControlByLocalId.Request[	**30**
	inputOutputLocalId = "Mask/Demask Troublecodes"	07
	inputOutputControlParameter = long Termadjust	08
	Byte LED-Output	xx
	Checksum	xx

time	Server Positive Response Message	Hex	Server Negative Response Message	Hex
P2	Format Byte	xx	Format Byte	xx
	inputOutputControlByLocalId.PosRspons	**70**	**negativeResponse Service Identifier**	**7F**
	[inputOutputLocalId = "Mask/Demask Troublecodes“	07	**inputOutputControlByLocalId.ReqSId** [	**30**
	inputOutputControlParameter = longTermadjust]	08	responseCode]	xx
	Checksum	xx	Checksum	xx

Table 10.17 Byte LED-Output

Hex value	description
Bit 0 (LSB) = 0	Demask troublecodes for LED-Output
Bit 0 (LSB) = 1	Mask troublecodes for LED-Output

Note Bit 1-7 are set to logic low.

Confidential

Table 10.18 Reset Immobiliser

time	Client Request Message	Hex
P3	Format Byte	05
	inputOutputControlByLocalId.Request[	**30**
	inputOutputLocalId = "securityCode"	62
	inputOutputControlParameter = resetImmobilizerToDeliveryCondition	0F
	securityCode { Byte 1 }	xx
	securityCode { Byte 2 }]	xx
	Checksum	xx

time	Server Positive Response Message	Hex	Server Negative Response Message	Hex
P2	Format Byte	03	Format Byte	83
	inputOutputControlByLocalId.PosRspons	**70**	**negativeResponse Service Identifier**	**7F**
	[inputOutputLocalId = "securityCode"	62	**inputOutputControlByLocalId.ReqSId** [	**30**
	inputOutputControlParameter = resetImmobilizerToDeliveryCondition]	0F	responseCode]	xx
	Checksum	xx	Checksum	xx

Table 10.19 Reset ECM

time	Client Request Message	Hex
P3	Format Byte	05
	inputOutputControlByLocalId.Request[	**30**
	inputOutputLocalId = "securityCode"	62
	inputOutputControlParameter = resetEcmToDeliveryCondition	0E
	securityCode { Byte 1 }	xx
	securityCode { Byte 2 }]	xx
	Checksum	xx

time	Server Positive Response Message	Hex	Server Negative Response Message	Hex
P2	Format Byte	03	Format Byte	03
	inputOutputControlByLocalId.PosRspons	**70**	**negativeResponse Service Identifier**	**7F**
	[inputOutputLocalId = "securityCode"	62	**inputOutputControlByLocalId.ReqSId** [	**30**
	inputOutputControlParameter = resetEcmToDeliveryCondition]	0E	responseCode]	xx
	Checksum	xx	Checksum	xx

Confidential

Table 10.20 Program Secret Key and Security Code into ECM

time	Client Request Message	Hex
P3	Format Byte	05
	inputOutputControlByLocalId.Request[	**30**
	inputOutputLocalId = "securityCode"	62
	inputOutputControlParameter = ecmLearnSecretKeyAndSecurityCode	0A
	securityCode { Byte 1 }	xx
	securityCode { Byte 2 }]	xx
	Checksum	xx

time	Server Positive Response Message	Hex
P2	Format Byte	03
	inputOutputControlByLocalId.PosRspons	**70**
	[inputOutputLocalId = "securityCode"	62
	inputOutputControlParameter = ecmLearnSecretKeyAndSecurityCode]	0A
	Checksum	xx

Server Negative Response Message	Hex
Format Byte	03
negativeResponse Service Identifier	**7F**
inputOutputControlByLocalId.ReqSId [	**30**
responseCode]	xx
Checksum	xx

Table 10.21 Program Secret Key from ECM

time	Client Request Message	Hex
P3	Format Byte	05
	inputOutputControlByLocalId.Request[	**30**
	inputOutputLocalId = "securityCode"	62
	inputOutputControlParameter = immoLearnSecretKey	0B
	securityCode { Byte 1 }	xx
	securityCode { Byte 2 }]	xx
	Checksum	xx

time	Server Positive Response Message	Hex
P2	Format Byte	03
	inputOutputControlByLocalId.PosRspons	**70**
	[inputOutputLocalId = "securityCode"	62
	inputOutputControlParameter = immoLearnSecretKey]	0B
	Checksum	xx

Server Negative Response Message	Hex
Format Byte	03
negativeResponse Service Identifier	**7F**
inputOutputControlByLocalId.ReqSId [	**30**
responseCode]	xx
Checksum	xx

Confidential

Table 10.22 Actuator test - LED-Output (Activate Actuatortest)

time	Client Request Message	Hex
P3	Format Byte	xx
	inputOutputControlByLocalId.Request[	**30**
	inputOutputLocalId = "Actuatortest LED-Output"	01
	inputOutputControlParameter = shortTermadjust	07
	Byte LED-Output	xx
	Checksum	xx

time	Server Positive Response Message	Hex	Server Negative Response Message	Hex
P2	Format Byte	xx	Format Byte	xx
	inputOutputControlByLocalId.PosRspons	**70**	**negativeResponse Service Identifier**	**7F**
	[inputOutputLocalId = "Actuatortest LED-Output“	01	**inputOutputControlByLocalId.ReqSId**	**30**
	inputOutputControlParameter = shortTermadjust]	07	[responseCode]	xx
	Checksum	xx	Checksum	xx

Table 10.23 Byte LED-Output

Hex value	description
Bit 0 (LSB) = 0	LED-Output deactive
Bit 0 (LSB) = 1	LED-Output active

Note Bit 1-7 are set to logic low.

Table 10.24 Actuator test - LED-Output (Return_Control_to_ECU)

time	Client Request Message	Hex
P3	Format Byte	xx
	inputOutputControlByLocalId.Request[	**30**
	inputOutputLocalId = "Actuatortest LED-Output"	01
	inputOutputControlParameter = returncontroltoECU	00
	Checksum	xx

time	Server Positive Response Message	Hex	Server Negative Response Message	Hex
P2	Format Byte	xx	Format Byte	xx
	inputOutputControlByLocalId.PosRspons	**70**	**negativeResponse Service Identifier**	**7F**
	[inputOutputLocalId = "Actuatortest LED-Output“	01	**inputOutputControlByLocalId.ReqSId** [	**30**
	inputOutputControlParameter = returncontroltoECU]	00	responseCode]	xx
	Checksum	xx	Checksum	xx

Confidential

Table 10.25 Actuator test - Read Transponder (Activate Actuatortest)

time	Client Request Message	Hex
P3	Format Byte	xx
	inputOutputControlByLocalId.Request[	**30**
	inputOutputLocalId = "Actuatortest Read Transponder"	03
	inputOutputControlParameter = shortTermadjust	07
	Byte Read Transponder	xx
	Checksum	xx

time	Server Positive Response Message	Hex	Server Negative Response Message	Hex
P2	Format Byte	xx	Format Byte	xx
	inputOutputControlByLocalId.PosRspons	**70**	**negativeResponse Service Identifier**	**7F**
	[inputOutputLocalId = "ActuatortestRead Transponder“	03	**inputOutputControlByLocalId.ReqSId** [	**30**
	inputOutputControlParameter = shortTermadjust]	07	responseCode]	xx
	Checksum	xx	Checksum	xx

Table 10.26 Byte Read Transponder

Hex value	description
Bit 1 = 0	Read Transponder and actualize snapshot and return control automaticaly to ECU afterwards

Note Bit 2-7 are set to logic low.

Table 10.27 Actuator test - Read Transponder (Return_Control_to_ECU)

time	Client Request Message	Hex
P3	Format Byte	xx
	inputOutputControlByLocalId.Request[	**30**
	inputOutputLocalId = "Actuatortest Read Transponder"	03
	inputOutputControlParameter = returncontroltoECU	00
	Checksum	xx

time	Server Positive Response Message	Hex	Server Negative Response Message	Hex
P2	Format Byte	xx	Format Byte	xx
	inputOutputControlByLocalId.PosRspons	**70**	**negativeResponse Service Identifier**	**7F**
	[inputOutputLocalId = "Actuatortest Read Transponder“	03	**inputOutputControlByLocalId.ReqSId** [	**30**
	inputOutputControlParameter = returncontroltoECU]	00	responseCode]	xx
	Checksum	xx	Checksum	xx

Confidential

11 Programming Functions Condition

Tabelle 11.1 Programming functions condition

No.	Programming Function description	Security Code protection	Possible with Ignition ON	Possible with Ignition Off
1	Program Security Code	X	X	X
2	Enter Security Code	X	X	X
3	Program Transponder (Fixcode and Secret Key)	X	X	programming functions not available
4	Program Transponder (Fixcode)	X	X	X
5	Erase Transponder (Fixcode)	X	X	X
6	Program Mechanical Keynumber	X	X	X
7	Program Vehicle Identification number	X	X	X
8	Program Repair shop / Tester serial number	X	X	X
9	Mask / Demask Troublecodes	X	X	X
10	Reset Immo	X	X	X
11	Reset ECM	X	X	programming functions not available
12	Program Secret Key and Security Code into ECM	X	X	
13	Program Secret Key from ECM	X	X	
14	Actuator tests	X	X	X

Confidential

Security Code protection means:

the function

- is available, when
  - the security code is not programmed or programmed and correctly entered
- is not available, when
  - the security code is programmed and not entered or incorrect entered (Wait-Period)

11.1 General

- In any case a request with a Service ID not mentioned in this document is received, the immobilizer responds with the negative response code „**Service not supported**“.
- In case a 1-byte header is expected by the immobilizer but the tester requests with a 3-byte header the immobiliser doesn´t respond with a negative responsecode.

11.2 Valid Requests and it’s Corresponding Negative Response Code

11.2.1 Clear Diagnostic Information

Tabelle 11.1 NR-Codes „Clear Diagnostic Information

Serv-ID	Contr. ID	NR-Code	Conditions
14	- -	78: Response pending	always
		31: Request out of range	Group of Diagnostic Information ≠ 0x8000 and ≠ 0xFF00

Confidential

11.2.2 Input Output Control By local Identifier

In any case an Input/Output Control Service is requested, the immobilizer first response the Negative Response „**Request Correctly Received - Response Pending**“. Afterwards the request is evaluated and a corresponding Response is prepared.
All the IOCBLI-Services (except „Enter Security Code“) are Security Code protected, i.e. if the Security Code is already programmed but not entered correctly during the diagnostic session, the immobilizer responds with the Neg. Resp. „**Security Code Not Entered**“.

The following negative response codes are further possible

Tabelle 11.2 NR-Codes „InputOutputControlbylocalIdentifier

Serv-ID	Contr. ID	NR-Code	Conditions / Comments
30	01 02 (actuator test)	31: Request out of range	„inputOutputControlParameter“ = 0x07 but „controlState“ > 1 **or** „inputOutputControlParameter“ unknown (unequal to 7 and 0)
	03 04 05 (Tran-sponder)	A4: Programming Conditions Violated	Transponder Programming function requested but no Transponder or no STEC security transponder could be detected (see Snapshot: Statusbyte Transponder) **or** all positions are already used (see Snapshot: Status programmed positions) **or** Transponder is already programmed (see Snapshot: Status position of last received fixcode) or Ignition is OFF during request
		31: Request out of range	Transponder Erasing function requested but parameter „Transponder Fixcode Position“ >0x05
	06	no additional NRCs	Program Mechanical Keynumber
	07	31: Request out of range	mask/de-mask trouble codes: parameter „Byte LED/Relay-Output “ > 0x03
	62 (Security Code)	XX: various	in case the ECM responds with a specific negative responsecodes during one of the programing functions this responsecode is passed from the immobiliser to the tester . See ECM documentation for further information.
		37: Required Time Delay not Expired	Security Code was entered wrong - time delay is still active
		40: Download not Accepted	Immobilizer Specific Programming Function was requested but Immo was not able to establish a valid communication to the ECM
		79: Incorrect Byte Count	Format Byte (length) ≠ 0x05
		A1: System not in delivery condition	Program Security Code Requested but Security Code is already programmed
		A2: Incorrect Security Code	entered Security Code is unequal to the internally stored
	90	no additional NRCs	Program Vehicle Ident Number
	98	no additional NRCs	Program Repair Shop Code
	else	12: Subfunction not Supported	

Confidential

11.2.3 Read Data By Local Identifier

Tabelle 11.3 NR-Codes „ReadDatabylocalIdentifier

Serv-ID	Contr. ID	NR-Code	Conditions
21	01	no additional NRCs	read snap shot
	03	no additional NRCs	read mechanical keynumber
	else	12: Subfunction not Supported	

11.2.4 Read Diagnostic Trouble Codes By Status

Tabelle 11.4 NR-Codes „Clear Diagnostic Information“

Serv-ID	Contr. ID	NR-Code	Conditions
18	- -	31: Request out of range	Group of Diagnostic Information ≠ 0x8000 and ≠ 0xFF00 **or** Status of DTC ≠ 0x00

11.2.5 Read ECU Identification

This request leads to the Negative Response „**Subfunction not Supported**“ in case the identification option is not defined within the diagnostic specification (e.g. all request with identification option < 0x81)

11.2.6 Start Communication

Neg. Response not provided because of KWP2000 conventions.

11.2.7 Start Communication

Only general negative response are provided.

11.2.8 Tester Present

Only general negative response are provided.

Confidential